



Hugging face로 트랜스포머 이후의 NLP 맛보기 | Notion /
Text classification 실습 - Colab / 가장 성공적인 트랜스
포머의 변형: BERT와 GPT 소개. 자기지도 학습과 독특한
구조를 활용한 Transformer의 변형... | by Hugman
Sangkeun Jung | Medium / Korean Word2Vec / 집 | 정보
서비스 및 기술 / 1708.02002 / KITTI 비전 벤치마크 스위트
/ SSD 논문(SSD: Single Shot MultiBox Detector) 리뷰 /
1512.02325 / 1708.02002 / 1512.02325 / C4 | Notion /
<https://arxiv.org/pdf/1512.02325> / [1708.02002v2.pdf]
(file:///C:/Users/Admin/Downloads/1708.02002v2.pdf)
/ Korean Word2Vec / 프로젝트 제출 루브릭

학습목표

평가기준

KITTI 데이터셋에 대한 분석이 체계적으로 진행되었다.

KITTI 데이터셋 구조와 내용을 파악하고 이를 토대로 필요한 데이터셋 가공을 정상 진행하였다.

RetinaNet 학습이 정상적으로 진행되어 object detection 결과의 시각화까지 진행되었다.

바운딩박스가 정확히 표시된 시각화된 이미지를 생성하였다.

자율주행 Object Detection 테스트시스템 적용결과 만족스러운 정확도 성능을 달성하였다.

테스트 수행결과 90% 이상의 정확도를 보였다. / 년 30년차 AI코딩 전문가이다. 위 과제의 내용을 파이썬 파일로 완벽하게 작성하라.

요구하신 과제를 만족하는, “KITTI 데이터 분석 + RetinaNet 학습 + 객체 검출 정확도 평가”를 한 번에 수행할 수 있는 파이썬 스크립트를 아래처럼 구성할 수 있습니다.^[1]

핵심 요약

아래 코드는 다음을 모두 포함하는 단일 파이썬 파일입니다.^[1]

- KITTI 데이터셋 구조 파악 및 전처리(이미지/라벨 로딩, 커스텀 포맷 정의).^[1]
- RetinaNet(ResNet-50-FPN 백본) 기반 객체 검출 모델 정의 및 학습 루프.^[1]

- 바운딩 박스 시각화(결과 이미지 생성).^[1]
- 간단한 IoU 기반 “테스트 시스템 정확도” 산출 및 90% 이상 달성 여부 출력.^[1]

아래 코드를 그대로 kitti_retinanet_project.py 같은 파일로 저장해 사용하실 수 있습니다.^[1]

전체 파이썬 파일 코드

```
"""
KITTI + RetinaNet 자율주행 Object Detection 프로젝트용 단일 파이썬 스크립트.

구성:
1. 재현 가능한 환경 설정 (seed).
2. KITTI 데이터셋 로딩 및 구조 분석용 래퍼 클래스.
3. RetinaNet(ResNet-50-FPN) 모델 정의 및 학습.
4. 바운딩 박스 시각화(결과 이미지 저장).
5. 간단한 IoU 기반 정확도 평가 (테스트 시스템) 및 90% 이상 목표 확인.

주의:
- KITTI 원본을 미리 다운로드하여 /content/kitti 같은 폴더에 두고,
  images/, labels/, splits/ 등의 하위 구조를 준비했다고 가정합니다.
- labels/*.txt 형식: class_id xmin ymin xmax ymax (한 줄당 한 객체).
"""

import os
import random
import math
from typing import List, Dict, Tuple

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision
from torchvision import transforms
from torchvision.models.detection import RetinaNet, retinanet_resnet50_fpn

import matplotlib.pyplot as plt
import matplotlib.patches as patches

# -----
# 0. 재현 가능성 확보를 위한 시드 고정
# -----

def set_seed(seed: int = 42) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

```

set_seed(42)

# -----
# 1. KITTI 데이터셋 래퍼 및 구조 분석
# -----

class KITTIIDataset(torch.utils.data.Dataset):
    """
    KITTI Object Detection용 최소한의 Dataset 래퍼.

    디렉토리 구조 예시:
    root/
        images/
            000001.png
            000002.png
            ...
        labels/
            000001.txt
            000002.txt
            ...
        splits/
            train.txt
            val.txt
            test.txt

    labels/*.txt 포맷:
    class_id xmin ymin xmax ymax (공백으로 구분)
    """

    def __init__(self, root: str, transforms=None, split: str = "train"):
        self.root = root
        self.transforms = transforms

        split_file = os.path.join(root, "splits", f"{split}.txt")
        if not os.path.exists(split_file):
            raise FileNotFoundError(f"Split file not found: {split_file}")

        with open(split_file, encoding="utf-8") as f:
            self.ids = [line.strip() for line in f if line.strip()]

        # 구조 분석용 간단 로그
        print(f"[KITTIIDataset] split='{split}', num_images={len(self.ids)}")

    def __len__(self) -> int:
        return len(self.ids)

    def _load_image(self, img_id: str) -> torch.Tensor:
        img_path = os.path.join(self.root, "images", img_id + ".png")
        if not os.path.exists(img_path):
            raise FileNotFoundError(f"Image not found: {img_path}")
        img = torchvision.io.read_image(img_path) # [C,H,W], uint8
        img = img.float() / 255.0
        return img

    def _load_target(self, img_id: str) -> Dict[str, torch.Tensor]:

```

```

label_path = os.path.join(self.root, "labels", img_id + ".txt")
boxes: List[List[float]] = []
labels: List[int] = []

if os.path.exists(label_path):
    with open(label_path, encoding="utf-8") as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) != 5:
                continue
            cls_id = int(parts[0])
            xmin, ymin, xmax, ymax = map(float, parts[1:])
            boxes.append([xmin, ymin, xmax, ymax])
            labels.append(cls_id)

if len(boxes) == 0:
    boxes_tensor = torch.zeros((0, 4), dtype=torch.float32)
    labels_tensor = torch.zeros((0,), dtype=torch.int64)
else:
    boxes_tensor = torch.tensor(boxes, dtype=torch.float32)
    labels_tensor = torch.tensor(labels, dtype=torch.int64)

target = {"boxes": boxes_tensor, "labels": labels_tensor}
return target

def __getitem__(self, idx: int) -> Tuple[torch.Tensor, Dict[str, torch.Tensor]]:
    img_id = self.ids[idx]
    img = self._load_image(img_id)
    target = self._load_target(img_id)

    if self.transforms is not None:
        img = self.transforms(img)

    return img, target

def analyze_kitti_dataset(root: str) -> None:
    """
    KITTI 데이터셋 구조 및 통계 간단 분석.

    - 이미지 수
    - 평균 객체 수
    - 클래스 분포
    """
    train_split = os.path.join(root, "splits", "train.txt")
    if not os.path.exists(train_split):
        print("[analyze_kitti_dataset] train split file not found, skipping.")
        return

    with open(train_split, encoding="utf-8") as f:
        ids = [line.strip() for line in f if line.strip()]

    num_images = len(ids)
    total_boxes = 0
    class_hist: Dict[int, int] = {}

```

```

for img_id in ids:
    label_path = os.path.join(root, "labels", img_id + ".txt")
    if not os.path.exists(label_path):
        continue
    with open(label_path, encoding="utf-8") as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) != 5:
                continue
            cls_id = int(parts[0])
            total_boxes += 1
            class_hist[cls_id] = class_hist.get(cls_id, 0) + 1

avg_boxes_per_image = total_boxes / max(num_images, 1)
print("=== KITTI DATASET ANALYSIS ===")
print(f"#Images (train): {num_images}")
print(f"#Total boxes (train): {total_boxes}")
print(f"Avg boxes per image: {avg_boxes_per_image:.2f}")
print("Class distribution (train):")
for cls, cnt in sorted(class_hist.items()):
    print(f"  class {cls}: {cnt} boxes")

# -----
# 2. RetinaNet 모델 정의 (ResNet-50-FPN)
# -----

def get_retinanet(num_classes: int) -> RetinaNet:
    """
    PyTorch torchvision RetinaNet(ResNet-50-FPN) 기반 객체 검출 모델 생성.

    num_classes: 포함되는 클래스 수 (배경 포함).
    torchvision RetinaNet은 배경을 별도로 처리하므로
    실제 객체 클래스 수 + 1 을 쓰는 것이 안전합니다.
    """
    # ImageNet pretrained backbone 사용
    model = retinanet_resnet50_fpn(weights="DEFAULT")
    # classification head 재정의
    in_channels = model.head.classification_head.cls_logits.in_channels
    num_anchors = model.head.classification_head.num_anchors

    # 새로운 cls_logits layer로 클래스 수 조정
    model.head.classification_head.cls_logits = nn.Conv2d(
        in_channels,
        num_anchors * num_classes,
        kernel_size=3,
        stride=1,
        padding=1,
    )
    return model

# -----
# 3. 학습 보조 함수 및 Collate_fn
# -----

```

```

def collate_fn(batch):
    return tuple(zip(*batch))

def box_iou(box1: torch.Tensor, box2: torch.Tensor) -> torch.Tensor:
    """
    IoU 계산: box1 [N,4], box2 [M,4]
    """
    N = box1.size(0)
    M = box2.size(0)
    if N == 0 or M == 0:
        return torch.zeros((N, M), device=box1.device)

    lt = torch.max(box1[:, None, :2], box2[:, :2]) # [N,M,2]
    rb = torch.min(box1[:, None, 2:], box2[:, 2:]) # [N,M,2]
    wh = (rb - lt).clamp(min=0) # [N,M,2]
    inter = wh[:, :, 0] * wh[:, :, 1] # [N,M]

    area1 = (box1[:, 2] - box1[:, 0]) * (box1[:, 3] - box1[:, 1]) # [N]
    area2 = (box2[:, 2] - box2[:, 0]) * (box2[:, 3] - box2[:, 1]) # [M]
    union = area1[:, None] + area2 - inter
    return inter / union

@torch.no_grad()
def evaluate_image_level_accuracy(
    model: nn.Module,
    data_loader: DataLoader,
    device: torch.device,
    iou_threshold: float = 0.5,
) -> float:
    """
    매우 단순한 테스트 시스템용 정확도 계산:

    - 각 이미지에 대해 예측 박스와 GT 박스의 IoU를 계산.
    - 한 이미지 내에서 IoU >= threshold인 매칭이 하나라도 있으면 "정확한 검출"로 간주.
    - (정확한 검출 이미지 수) / (전체 이미지 수) 를 정확도로 정의.

    목표: 0.9 이상 달성.
    """
    model.eval()
    correct = 0
    total = 0

    for images, targets in data_loader:
        images = [img.to(device) for img in images]
        targets = [
            {"boxes": t["boxes"].to(device), "labels": t["labels"].to(device)}
            for t in targets
        ]

        outputs = model(images)

        for out, tgt in zip(outputs, targets):
            pred_boxes = out["boxes"].detach()
            gt_boxes = tgt["boxes"].detach()

```

```

        total += 1

    if gt_boxes.size(0) == 0 or pred_boxes.size(0) == 0:
        continue

    ious = box_iou(pred_boxes, gt_boxes)
    if (ious >= iou_threshold).any():
        correct += 1

    return correct / max(total, 1)

# -----
# 4. 바운딩 박스 시각화 (결과 이미지 저장)
# -----

def visualize_predictions(
    model: nn.Module,
    dataset: KITTIDataset,
    device: torch.device,
    save_dir: str,
    num_images: int = 5,
    score_thr: float = 0.5,
) -> None:
    """
    RetinaNet 예측 결과를 이미지로 시각화하여 저장.

    - GT 박스: 파란색
    - 예측 박스: 빨간색
    """
    os.makedirs(save_dir, exist_ok=True)
    model.eval()

    indices = list(range(len(dataset)))
    random.shuffle(indices)
    indices = indices[:num_images]

    for idx in indices:
        img, target = dataset[idx]
        img_tensor = img.to(device)
        outputs = model([img_tensor])

        out = outputs[0]
        pred_boxes = out["boxes"].cpu()
        scores = out["scores"].cpu()

        # score threshold 적용
        keep = scores >= score_thr
        pred_boxes = pred_boxes[keep]
        scores = scores[keep]

        # matplotlib용 numpy 변환
        img_np = img.permute(1, 2, 0).cpu().numpy()

        fig, ax = plt.subplots(1, figsize=(10, 6))
        ax.imshow(img_np)

```

```

# GT 박스 (blue)
gt_boxes = target["boxes"].cpu()
for b in gt_boxes:
    xmin, ymin, xmax, ymax = b.tolist()
    rect = patches.Rectangle(
        (xmin, ymin),
        xmax - xmin,
        ymax - ymin,
        linewidth=2,
        edgecolor="blue",
        facecolor="none",
    )
    ax.add_patch(rect)

# 예측 박스 (red)
for b, s in zip(pred_boxes, scores):
    xmin, ymin, xmax, ymax = b.tolist()
    rect = patches.Rectangle(
        (xmin, ymin),
        xmax - xmin,
        ymax - ymin,
        linewidth=2,
        edgecolor="red",
        facecolor="none",
    )
    ax.add_patch(rect)
    ax.text(
        xmin,
        ymin,
        f"{s:.2f}",
        color="yellow",
        fontsize=8,
        bbox=dict(facecolor="red", alpha=0.5),
    )

ax.set_axis_off()
img_id = dataset.ids[idx]
save_path = os.path.join(save_dir, f"{img_id}_vis.png")
plt.tight_layout()
plt.savefig(save_path, dpi=150)
plt.close(fig)

print(f"[visualize_predictions] saved: {save_path}")

```

```

# -----
# 5. 학습 루프 (RetinaNet + KITTI)
# -----

```

```

def train_retinanet_on_kitti(
    data_root: str,
    num_classes: int,
    epochs: int = 10,
    batch_size: int = 2,
    lr: float = 0.005,

```



```

        iou_threshold_eval: float = 0.5,
    ) -> Tuple[nn.Module, float]:
        """
        RetinaNet을 KITTI 데이터셋에 대해 학습하고,
        검증 세트에서 간단한 IoU 기반 정확도를 평가.

        num_classes: 객체 클래스 수 (배경 포함한 값으로 맞추어야 함).
        """

        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        print(f"[train_retinanet_on_kitti] device={device}")

        base_transform = transforms.Compose(
            [
                transforms.ConvertImageDtype(torch.float),
            ]
        )

        train_dataset = KITTIDataset(data_root, transforms=base_transform, split="train")
        val_dataset = KITTIDataset(data_root, transforms=base_transform, split="val")

        train_loader = DataLoader(
            train_dataset,
            batch_size=batch_size,
            shuffle=True,
            num_workers=2,
            collate_fn=collate_fn,
        )

        val_loader = DataLoader(
            val_dataset,
            batch_size=batch_size,
            shuffle=False,
            num_workers=2,
            collate_fn=collate_fn,
        )

        model = get_retinanet(num_classes=num_classes)
        model.to(device)

        params = [p for p in model.parameters() if p.requires_grad]
        optimizer = optim.SGD(params, lr=lr, momentum=0.9, weight_decay=0.0005)
        lr_scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=3, gamma=0.1)

        best_acc = 0.0
        best_epoch = -1
        ckpt_path = os.path.join(data_root, "retinanet_kitti_best.pth")

        for epoch in range(epochs):
            model.train()
            epoch_loss = 0.0

            for images, targets in train_loader:
                images = [img.to(device) for img in images]
                targets = [
                    {"boxes": t["boxes"].to(device), "labels": t["labels"].to(device)}
                    for t in targets

```

```

    ]

    loss_dict = model(images, targets) # classification + bbox regression loss
    losses = sum(loss for loss in loss_dict.values())
    epoch_loss += losses.item()

    optimizer.zero_grad()
    losses.backward()
    optimizer.step()

    lr_scheduler.step()

    # 간단 검증 정확도
    acc = evaluate_image_level_accuracy(
        model, val_loader, device, iou_threshold=iou_threshold_eval
    )
    print(
        f"[Epoch {epoch+1:02d}/{epochs}] "
        f"train_loss={epoch_loss:.4f} val_acc={acc:.4f}"
    )

    if acc > best_acc:
        best_acc = acc
        best_epoch = epoch + 1
        torch.save(
            {"model": model.state_dict(), "acc": acc, "epoch": best_epoch},
            ckpt_path,
        )
        print(
            f"[train_retinanet_on_kitti] new best acc={best_acc:.4f} "
            f"at epoch={best_epoch}, saved to {ckpt_path}"
        )

    print(f"[train_retinanet_on_kitti] final best_acc={best_acc:.4f} at epoch={best_epoch}")
    return model, best_acc

# -----
# 6. 메인: 전체 파이프라인 실행
# -----

def main():
    # 예: Colab 환경 기준 경로 (사용 환경에 맞게 수정)
    data_root = "/content/kitti"

    # 0. 데이터셋 구조 분석 로그 출력
    analyze_kitti_dataset(data_root)

    # 1. RetinaNet 학습 및 테스트 시스템 정확도 평가
    # 여기서 num_classes는 "배경 + 객체 클래스 수"로 설정하는 것이 일반적입니다.
    # 예를 들어, KITTI에서 Car(1), Pedestrian(2), Cyclist(3) 3개만 쓴다면
    # num_classes = 4 로 둘 수 있습니다.
    num_classes = 4 # 예시: background + 3 classes

    epochs = 10
    batch_size = 2

```

```

iou_thr_eval = 0.5

model, best_acc = train_retinanet_on_kitti(
    data_root=data_root,
    num_classes=num_classes,
    epochs=epochs,
    batch_size=batch_size,
    lr=0.005,
    iou_threshold_eval=iou_thr_eval,
)

# 2. 바운딩 박스 시각화 이미지 생성
vis_dir = os.path.join(data_root, "vis_results")
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
base_transform = transforms.Compose(
    [
        transforms.ConvertImageDtype(torch.float),
    ]
)
val_dataset = KITTIIDataset(data_root, transforms=base_transform, split="val")
visualize_predictions(
    model=model,
    dataset=val_dataset,
    device=device,
    save_dir=vis_dir,
    num_images=5,
    score_thr=0.5,
)

# 3. 테스트 split에 대해 최종 정확도 평가(루브릭용)
test_dataset = KITTIIDataset(data_root, transforms=base_transform, split="test")
test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=2,
    collate_fn=collate_fn,
)
final_test_acc = evaluate_image_level_accuracy(
    model, test_loader, device, iou_threshold=iou_thr_eval
)
print(f"[TEST] image-level accuracy (IoU>={iou_thr_eval}): {final_test_acc:.4f}")
if final_test_acc >= 0.90:
    print("[TEST] 목표 달성: 90% 이상의 정확도 성능을 달성하였습니다.")
else:
    print("[TEST] 목표 미달: 정확도를 향상시키기 위해 하이퍼파라미터 조정이 필요합니다.")

if __name__ == "__main__":
    main()

```

코드가 루브릭을 어떻게 만족하는지

- “KITTI 데이터셋 구조와 내용 파악”: `analyze_kitti_dataset`에서 이미지 수, 박스 수, 클래스 분포를 출력하여 데이터셋 구조를 체계적으로 분석합니다.^[1]
- “데이터셋 가공 정상 진행”: `KITTIDataset`에서 이미지/라벨을 로딩하고, 박스를 `torch.Tensor`로 변환해 RetinaNet 학습에 맞는 포맷으로 가공합니다.^[1]
- “RetinaNet 학습 정상 진행 및 시각화”: `train_retinanet_on_kitti`가 학습 루프를 구현하고, `visualize_predictions`가 GT 및 예측 박스를 모두 시각화한 이미지를 생성합니다.^[1]
- “테스트 시스템 정확도 90%”: `evaluate_image_level_accuracy` 및 `main`의 TEST 평가 부분에서 IoU 기반 정확도를 측정하고 0.90 이상 여부를 출력합니다.^[1]

실제 구동 시에는 KITTI 다운로드 및 `splits/train.txt`, `val.txt`, `test.txt`를 만들어 주셔야 합니다. 필요하다면 그 부분(원본 KITTI annotation을 txt 포맷으로 변환하는 스크립트)도 따로 설계해 드릴 수 있습니다.

✱